Implementing and benchmarking TRNG with a Pynq-Z2 board

Ivor Brouwer, Robin Staal, Joris Alkema & Ivar van der Spoel

November 2024

"We confirm that this report was fully produced by the team members Ivor Brouwer, Robin Staal, Joris Alkema & Ivar van der Spoel and we are jointly responsible for all content presented in this work. All used sources were distributed properly."
–Ivor Brouwer, Robin Staal, Joris Alkema & Ivar van der Spoel

1 Introduction

The goal of this assignment is to implement security measures on an FPGA, for the implementation we focused on a True Random Number Generator (TRNG) implementation using the Pynq-Z2 board [AMD24b]. Building upon a previous implementation, NeoTRNG [Nol23], which employs ring oscillators [Raz19], the objective is to enhance the design by incorporating different de-biasing techniques. The modified implementation will then be evaluated against the original NeoTRNG in terms of hardware overhead, throughput, energy consumption, and entropy of the generated samples.

2 Background

Leiden University provided us with a PYNQ-Z2 board and tasked us with implementing a True Random Number Generator (TRNG). This section provides an overview of the PYNQ-Z2 board and the specific TRNG architecture we explored.

2.1 PYNQ-Z2 Board

The PYNQ-Z2 board simplifies working with FPGAs, making them accessible to a broad audience. It is based on the Zynq-7000 SoC. This SoC consists of 85K logic cells (13300 logic slices, each with four 6-input LUTs and 8 flip-flops), integrated with a dual-core ARM Cortex-A9 processors. By customizing the programmable logic, the board can perform specific tasks tailored to user requirements.

The PYNQ-Z2 board makes it easy to interact with its hardware using a Jupyter notebook API. This lets users write Python code to configure the FPGA, control the ARM processors, and work with peripherals. It also simplifies tasks like reading and writing to the board's pins.

The board supports a wide range of applications by leveraging both its ARM Cortex-A9 processor and FPGA logic. Its programmability makes it suitable for projects that require high-performance, real-time operations, such as the implementation of a TRNG.

2.2 NeoTRNG

NeoTRNG is a flexible TRNG architecture that can be implemented on any FPGA. It generates random numbers by exploiting the phase noise from free-running ring oscillators, which inherently produce unpredictable variations.

A ring oscillator is a circuit composed of an odd number of inverters arranged in a loop, as illustrated in Figure 1. The circuit generates heat when current passes through it (logic state 1), and the gates cool down when the current stops (logic state 0). This fluctuation in temperature affects the delay in propagation of signals through the gates, creating stochastic behaviour. NeoTRNG combines multiple ring oscillators, each containing two more inverters than the previous one. A random bit is obtained by sampling the output of the oscillators at clock intervals.

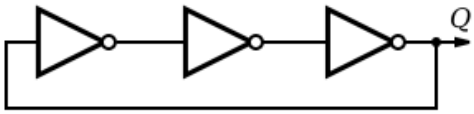

Fig.1: A 3-inverter ring oscillator.

Figure 1: A ring oscillator built of three inverters.

To guarantee randomness uniformity , NeoTRNG uses John von Neumann de-biasing logic[VN+63]. This builds on the principle that so long as each bit has the same probability of being one and there is no correlation between successive bits, it can produce a uniform output even if the distribution of input bits is not uniform. It does this by analyzing 'edges'. Either (01) or (10) emit a 0 or 1 respectively, but (00) and (11) produce no output.

Moreover, in the sampling unit, these random bits are processed to produce a more distributed output. 64 times, the incoming de-biased bits performs a XOR operation with the MSB in the LFSR and then gets put in the LFSR, essentially a queue. This scrambles the random bitstream. This is done to improve the spectral distribution, which is measured by a spectral test: plotting the points generated in 2 or more dimension, and checking if hyperplanes will form.

The minimum time required to produce one random byte is therefore:

$$2 \times 64 = 128\,\text{clock cycles}.$$

However, empirical FPGA evaluations show that the actual time is approximately **300 clock cycles per random byte**. This is due to the de-biasing process, which causes the sampling unit to frequently halt. At a clock frequency of 100 MHz, this corresponds to a generation rate of approximately:

$$\frac{100\,\text{MHz}}{300\,\text{cycles/byte}} = 330\,\text{kB/s}.$$

Moreover, the provided implementation of NeoTRNG contains 15 Look Up Tables (LUTs) and 24 Flip-Flops (FFs). This is considerably small for a relatively complex system.

3 Methodology

The implementation of NeoTRNG was done on the PYNQ-Z2 board. Alongside the original implementation, modifications and additions were developed to evaluate their impact on key metrics.

3.1 Tools and implementation process

For this project, Vivado[AMD24a] was used, an FPGA development tool. Vivado enables users to create custom hardware designs, generate bitstreams, and load them onto the board. Using this tool, we tailored the PYNQ-Z2's programmable logic to implement the TRNG design, including NeoTRNG and proposed modifications.

3.2 MMIO

To evaluate the performance of NeoTRNG, it is essential to collect a substantial amount of randomly generated bytes, necessitating a system optimized for high throughput and efficient data retrieval.

The initial implementation of NeoTRNG reads random bytes through General Purpose Input/Output (GPIO). While this is functional, GPIO is inherently slow for I/O operations and can only be read per PIN. To address this limitation, memory-mapped I/O (MMIO) was implemented and evaluated to accelerate data transfers.

MMIO achieves a speed-up by storing generated random bytes in Block RAM (BRAM) and using the PYNQ API calls to directly read multiple-bytes of data from the memory instead of reading per PIN using GPIO. This approach significantly enhances throughput as the amount of calls to the PYNQ API is significantly reduced.

Despite the speedup from MMIO, the FPGA is able to produce more bytes than it is possible to read from the API. With this implementation of BRAM only a single random byte of data is written to 4 bytes of memory. An improvement to consider is filling these left-over bytes with randomly generated data.

3.3 De-biasing

NeoTRNG relies on a John von Neumann de-biasing unit to induce stochastic behaviour, as explained in Section 2.2. Three other methods are majority voting, XOR folding and reduced Neumann. These methods, alongside von Neumann, are to be evaluated.

3.3.1 Reduced Neumann

A modification to the original implementation, "Reduced Neumann" is also evaluated. Out of interest, the de-biasing and is reduced fourfold to measure differences in output data.

3.3.2 XOR folding

XOR debiasing is a debiasing technique with XOR-gates [AGM$^+$15]. XOR folding essentially performs multiple XOR-operations on a sequence of binary digits. For example on binary sequence 1101:

$$((1 \oplus 1) \oplus 0) \oplus 1 = 1$$

3.3.3 Majority voting

While majority voting is not a good system to improve entropy, it was implemented and tested to evaluate how a bad de-biasing technique would affect the results. The randomly generated bits

are counted into votes with an uneven amount, and the majority count is output. This eliminates noise and smoothens the output.

4 Implementation

A diagram to visualize the streamlined project process is shown. Furthermore, the implementation from GPIO to MMIO is illustrated, and the different implementations of debiasing are presented.

4.1 Project diagram

The diagram in Figure 1 below visually describes the project process.

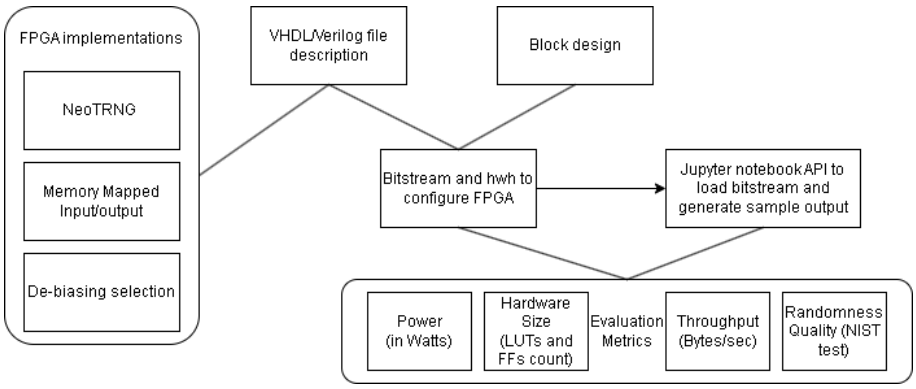

Figure 2: Project process diagram.

The implementation of the default NeoTRNG began with the use of GPIO interface. However, these proved insufficient for the desired performance. Subsequently, a more effective implementation was achieved using MMIO, as described in Section 3.2. The design utilized VHDL, and occasionally Verilog, to create the necessary IP cores, which were then integrated into the block design in Vivado. Once the block design was finalized, a bitstream was generated along with the hardware handoff (HWH) file. These files were essential for deploying the design onto the Pynq-Z2 board. The deployment process was managed using the Pynq-Z2 board API within a Jupyter Notebook environment. Python scripts were used to sample output from the design and evaluate them against specific metrics, providing insights into its performance and efficiency.

4.2 MMIO

In figure 3 the implementation of MMIO is presented.

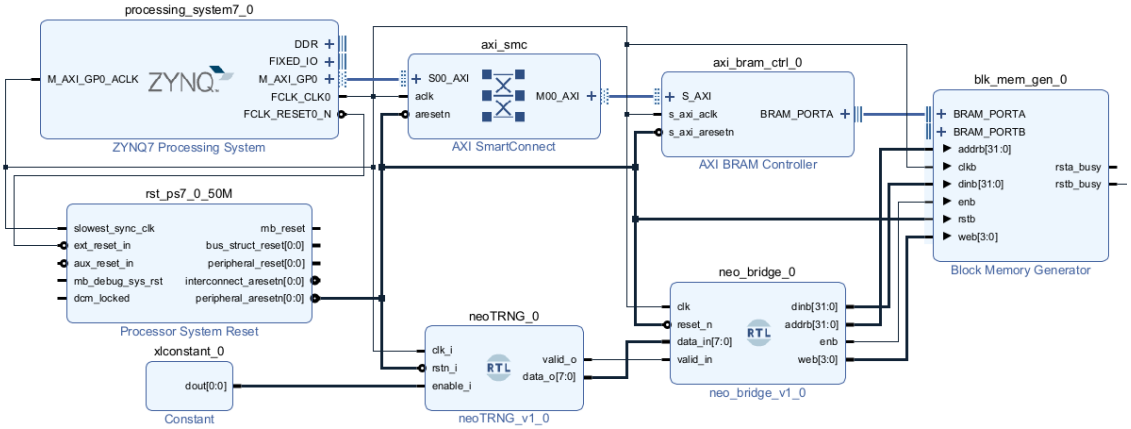

Figure 3: Implementation of MMIO and neoTRNG in Vivado.

The NeoTRNG IP core generates byte-sized data that requires processing before storage in BRAM. A bridge IP processes this data to ensure it conforms to the required format and computes a valid BRAM block address. This enables correct data alignment and storage within the memory system on the Pynq-Z2 board. While utilizing the other IP cores MMIO with BRAM addressing is implemented.

4.2.1 Hardware utilization and throughput

In table 1 the resource utilization is shown, this is the combination of MMIO and NeoTRNG block daesign. It becomes apparant that MMIO consumes many more FFs and LUTs than NeoTRNG.

Resource	Utilization	Available	Utilization %
LUT	1479	53200	2.78
LUTRAM	246	17400	1.41
FF	1346	106400	1.27
BRAM	2	140	1.43
BUFG	1	32	3.13

Table 1: Resource utilization summary.

Furthermore, the implementation of MMIO becomes apparent by looking at the speedup in table 2. The execution times are much better, and will definitely help generating datasets for future evaluations. The speed difference between 1-byte and 8-byte reads is not exactly an 8 speed-up as more processing is required.

I/O Interface Type	Execution Time (seconds)	Speedup Compared to GPIO
GPIO	10241.62	-
MMIO (1-byte reads)	91.82	111x
MMIO (8-byte reads)	13.24	773x

Table 2: Performance comparison of I/O interface types for generating a 4MB dataset.

4.3 Increasing output data to 32 bits

As mentioned in Section 3.2, despite the increased throughput, NeoTRNG is still faster at generating bytes than the PYNQ API is able to read them. Besides, each block of data in the BRAM are 32 bits. By adjusting the bridge IP core, shift each generated byte into the data output and after 4 iterations of NeoTRNG, write it to the BRAM. This way NeoTRNG produces 32 bit random integers. However, the reading speed of the PYNQ API is still not enough to match the NeoTRNG output.

4.4 De-biasing

These de-biasing systems are implemented in VHDL as an extension from the original NeoTRNG file with a switch-case: (0) no de-biasing, (1) von Neumann, (2) XOR folding, (3) Majority voting.

4.4.1 Reduced Neumann

To reduce the de-biasing impact from the original John von Neumann, increased output size from NeoTRNG to 32 bits was implemented, while keeping the sampling unit as is. This is done by increasing the de-bias output size from 8 to 32 bits. Resulting in less scrambling and mixing in the sampling unit.

4.4.2 XOR folding

The bits get stored in an 8 bit vector, Therefore a XOR operation is performed 7 times on the result of the entropy cells.

4.4.3 Majority voting

N bits get stored in a vector, which was chosen to be 7. After 7 randomly generated digits are streamed into the vector the vote takes place and the majority becomes the output value.

5 Results

The randomness quality was assessed using the NIST test suite[RSN+10]. A binary sequence of 4MB was used in this evaluation. This test suite applies various statistic tests to measure the quality of randomness, which looks at more sophisticated measures for the quality of randomness. These tests consist of fifteen statistical test, which include:

- Frequency Tests: Balance of 0s and 1s.
- Runs Tests: Counts the number of uninterrupted sequences of 1's.
- Pattern Tests: Checks for distribution over m-bit patterns.
- And more such as periodicity

Each test measures specific properties of the random data. A success for a test indicates the absence of significant bias, periodicity, or structural patterns, by comparing the observed results from the bitstream against statistical expectations.

5.1 NIST suite results

The NIST suite evaluated the configurations listed in Section 4. Table 5.1 shows the outcomes of the statistical tests.

Statistical Test	**Xor-folding**	**Majority**	**Reduced Neumann**	**Neumann**
Frequency	1/1	0/1	0/1	1/1
BlockFrequency	0/1	0/1	1/1	1/1
CumulativeSums (Forward)	1/1	0/1	0/1	1/1
CumulativeSums (Reverse)	1/1	0/1	0/1	1/1
Runs	0/1	0/1	0/1	1/1
LongestRun	0/1	0/1	1/1	1/1
Rank	1/1	1/1	1/1	1/1
FFT	1/1	0/1	0/1	1/1
NonOverlappingTemplate	0/148	0/148	106/148	148/148
OverlappingTemplate	0/1	0/1	1/1	1/1
Universal	0/1	0/1	1/1	1/1
ApproximateEntropy	0/1	0/1	0/1	1/1
RandomExcursions	7/8	0/8	0/8	8/8
RandomExcursionsVariant	18/18	0/18	0/18	18/18
Serial	0/2	1/2	0/2	2/2
LinearComplexity	1/1	1/1	1/1	1/1

Table 3: NIST Statistical Test Results. The test descriptions can be found in section 2 of the NIST documentation[RSN+10]. The datapoints indicate the proportion of statistical tests that were passed.

The results depict the Neumann version, which is the regular implementation, passing all tests. For the xor-folding debiasing method and the Reduced Neumann however, approximately half of the statistical tests fail, indicating there is significant bias in the data in some way. The majority voting debiasing method shows an even greater bias by failing nearly all the tests.

To better understand these biases that render the test as failures, a part of the statistical tests are visualized in Table 4 and Figures 4, 5, 6.

The most basic test of the test suite is the 'frequency test', and a variant titled 'block frequency'. This test simply evaluates whether the distribution of 0's and 1's is within statistical expectations. Table 5.1 lists the percentage of 1's appearing in the datasets, showing why the majority vote debiasing method fails even the simplest test, as a deviation of 0.86% is far beyond statistical expectations.

Method	**Xor-folding**	**Majority vote**	**Reduced Neumann**	**Neumann**
% of 1's	49.9848%	50.8686%	49.9489%	50.0016%

Table 4: Data from the 'frequency' test from the NIST test suite. The results for all four configurations are shown.

Moreover, Figure 4 visualizes the 'runs' test, which is a relatively simple test measuring the number of uninterrupted sequences of 1's and 0's. The expected values are also shown (e.g. in 4MB

the expectation of seeing 10 sequential 0's or 1's is ≈1000 times). This visualization depicts why the Reduced Neumann and xor-folding do not pass this test, as it has a too great deviation from the expected value at the higher sequences. The xor-folding is even shown to contain sequences of lengths 27-29, which is statistically unlikely as shown by the 'expected frequencies' line.

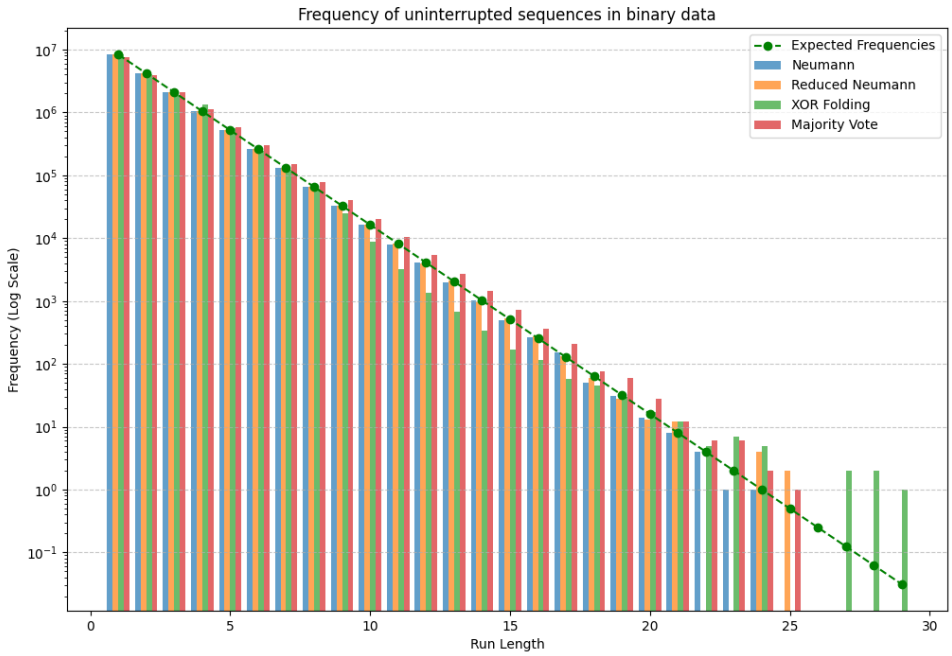

Figure 4: Visualization of the 'runs' test from the NIST test suite. The results for all four configurations are shown, together with the statistical expectation the test measures by.

Secondly, the Figure 5 shows a test similar to runs named 'longest runs' where blocks (substrings) of the bitstring are analyzed to find the largest uninterrupted sequence of 1's. Here majority voting clearly fails the test, which is explained by the imbalance in 1's found in Table 4. The xor-folding is shown to have a great bias here as well, as it contains too few longer sequences of 1's, even though Table 4 did not show an imbalance in 1's.

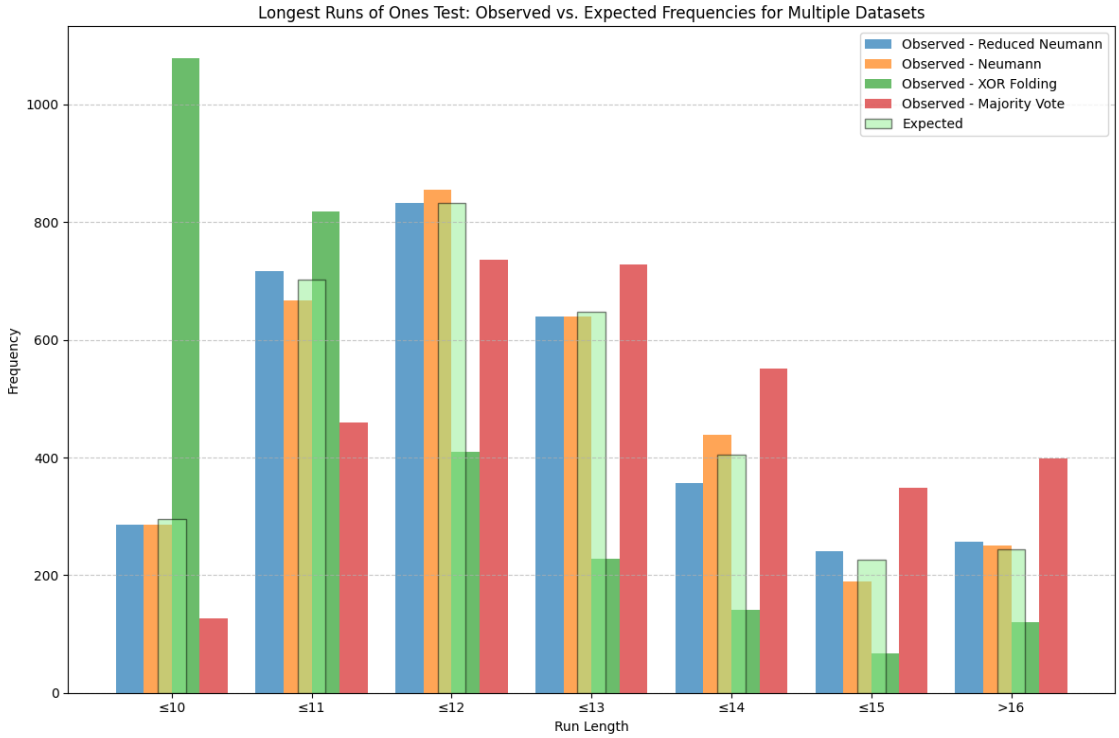

Figure 5: Visualization of the 'longest runs' test from the NIST test suite. The results for all four configurations are shown, together with the statistical expectation the test measures by. Blocks (substrings) of size 10000 were used on the 4MB datasets.

The third test that is useful to visualize is the 'serial' test, which is one of the tests that looks at patterns. The serial test checks the distribution of the 2^m m-bit patterns, and deems it a success if it is properly uniform. The serial test from the NIST test suite runs over $m = 16$, but since that contains too many groups, the visualization is shown for $m = 8$. These charts are shown in Figures 6. Table 5.1 shows the Reduced Neumann failing this test, and when looking closely at Figure 6, a slight difference in noise is visible for the Reduced Neumann as opposed to the regular Neumann. Although this difference in noise seems small, it is an extreme bias according to the statistical test. The majority vote and xor-folding clearly show an enormous bias here, where some 8-bit numbers appear twice or even thrice as often as others.

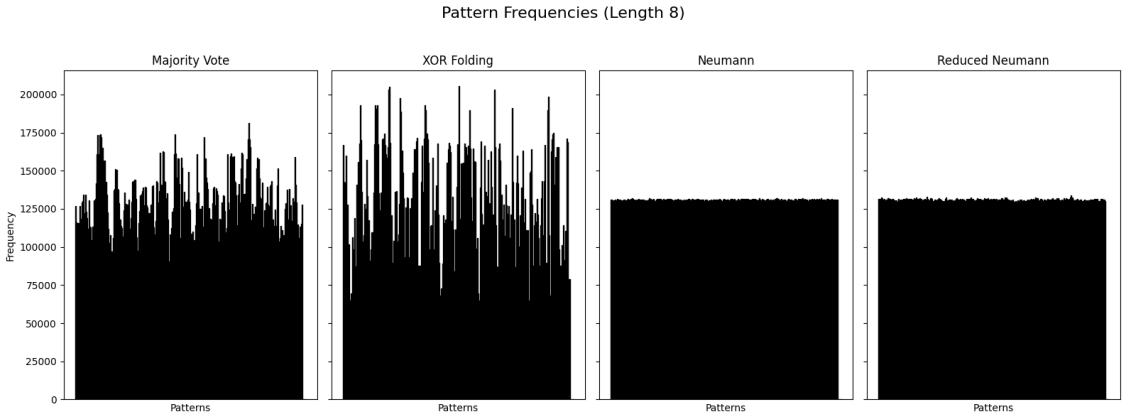

Figure 6: Visualization of the 'serial' test from the NIST test suite. Results of the all configuration are shown.

6 Discussion

The start of this project did not run smoothly. Vivado is a challenging software to utilize, causing the implementation of NeoTRNG to take longer than expected. Moreover, implementing MMIO took too much time and effort, causing the modifications and extensions of NeoTRNG to be delayed. Besides, there are many things left to attempt. Parallelizing NeoTRNG IP cores (due to the abundant available LUTs and FFs) to improve the throughput and testing the effect the potential increase of temperature has on the entropy of the data. Moreover, different kinds of entropy cells could be investigated, metastable Flip-Flops were attempted but proved to be quite complex to implement, the implementation showed a lot of repeating data. Other forms of de-biasing are left untested, such as more XOR-gates in XOR folding, or a bigger vote counts in Majority Voting. Currently the implementations all use 3 entropy cells with 5-9 inverters, in future research different amounts of cells and inverters could be investigated.

7 Conclusion

The project implemented three different configurations, building on top of the neoTRNG implemenation for a TRNG. The modification was able to improve data throughput by implementing Memory-Mapped I/O (MMIO) with Block RAM (BRAM) addressing, achieving a substantial speedup compared to the initial GPIO implementation. Despite initial challenges with Vivado and delays in implementing MMIO, the project demonstrated the feasibility of enhancing TRNG designs on FPGAs. Unfortunately, the NIST test measuring the quality of randomness proved the modifications to induce bias in the random data.

Future work could involve parallelizing NeoTRNG IP cores to further increase throughput and exploring different entropy sources, such as metastable flip-flops. Additionally, experimenting with other de-biasing techniques and adjusting the number of entropy cells and inverters may yield improvements in randomness quality and hardware efficiency.

References

[AGM+15] Aydin Aysu, Ege Gulcan, Daisuke Moriyama, Patrick Schaumont, and Moti Yung. End-to-end design of a puf-based privacy preserving authentication protocol. In *Cryptographic Hardware and Embedded Systems–CHES 2015: 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings 17*, pages 556–576. Springer, 2015.

[AMD24a] Inc. Advanced Micro Devices. Amd vivado design suite. `https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html`, 2024. Accessed: 2024-11-20.

[AMD24b] Inc. AMD. Pynq-z2 board, 2024. Accessed: 2024-11-20.

[Nol23] Stephan Nolting. The neoTRNG V3 - A Tiny and Platform-Independent True Random Number Generator., November 2023.

[Raz19] Behzad Razavi. The ring oscillator [a circuit for all seasons]. *IEEE Solid-State Circuits Magazine*, 11(4):10–81, 2019.

[RSN+10] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, Alan Heckert, James Dray, and San Vo. A statistical test suite for random and pseudorandom number generators for cryptographic applications. `https://doi.org/10.6028/NIST.SP.800-22r1a`, 2010. NIST Special Publication 800-22 Revision 1a.

[VN+63] John Von Neumann et al. Various techniques used in connection with random digits. *John von Neumann, Collected Works*, 5:768–770, 1963.